

Burak Demirci

2201292

COP4493 - AWS Academy Cloud Computing Fundamentals

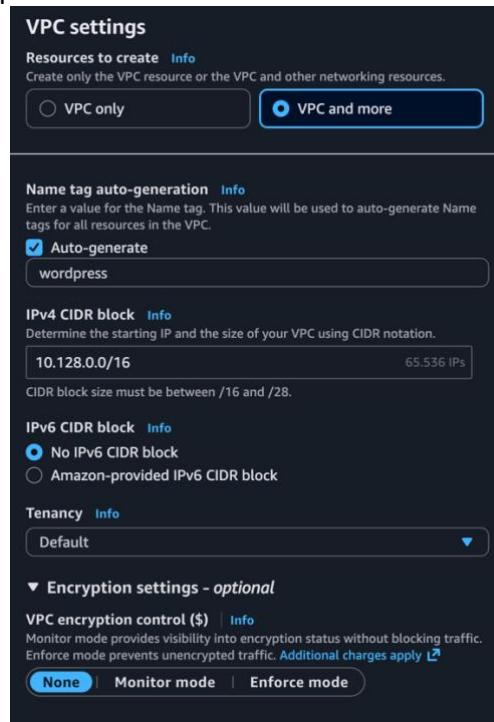
Project-2

URL of Website: <http://wordpress-529336691.us-east-1.elb.amazonaws.com>

AWS Account ID: 091781442554

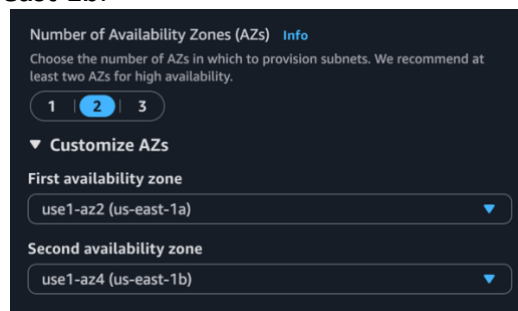
VPC Creation

1. **Resources to Create:** Select “VPC and more”. So, while creating VPC, NAT and subnets are also created by AWS.
2. **Name Tag Auto-Generation:** Enable “Auto-generate” and name anyway you wish, I named myself “wordpress”.
3. **IPv4 CIDR Block:** I gave the address space as “10.128.0.0/16”. But any other suitable spaces can be given.
4. **IPv6 CIDR Block:** Select “No IPv6 CIDR Block”.
5. **Tenancy:** Can be kept in “Default”.



The screenshot shows the 'VPC settings' page in the AWS console. It includes several sections: 'Resources to create' with radio buttons for 'VPC only' and 'VPC and more' (selected); 'Name tag auto-generation' with a checked 'Auto-generate' checkbox and a text input field containing 'wordpress'; 'IPv4 CIDR block' with a text input field containing '10.128.0.0/16' and a note that the CIDR block size must be between /16 and /28; 'IPv6 CIDR block' with radio buttons for 'No IPv6 CIDR block' (selected) and 'Amazon-provided IPv6 CIDR block'; 'Tenancy' with a dropdown menu set to 'Default'; and 'Encryption settings - optional' with buttons for 'None', 'Monitor mode', and 'Enforce mode'.

6. **Number of Availability Zones (AZs):** Select 2. And choose two different AZs, I choose **us-east-1a** and **us-east-1b**.



The screenshot shows the 'Number of Availability Zones (AZs)' section in the AWS console. It includes a section for 'Customize AZs' with two dropdown menus. The 'First availability zone' is set to 'use1-az2 (us-east-1a)' and the 'Second availability zone' is set to 'use1-az4 (us-east-1b)'.

7. **Number of Public/Private Subnets:** Select 2 for both public and private subnets.
8. **Subnets CIDR Blocks:** Is optional to modify. I named;
 - a. **Public Subnet CIDR Block in us-east-1a:** 10.128.0.0/24
 - b. **Public Subnet CIDR Block in us-east-1b:** 10.128.2.0/24
 - c. **Private Subnet CIDR Block in us-east-1a:** 10.128.1.0/24
 - d. **Private Subnet CIDR Block in us-east-1b:** 10.128.3.0/24

Number of public subnets [Info](#)
The number of public subnets to add to your VPC. Use public subnets for web applications that need to be publicly accessible over the internet.

0 2

Number of private subnets [Info](#)
The number of private subnets to add to your VPC. Use private subnets to secure backend resources that don't need public access.

0 2 4

▼ **Customize subnets CIDR blocks**

Public subnet CIDR block in us-east-1a

10.128.0.0/24 256 IPs

Public subnet CIDR block in us-east-1b

10.128.2.0/24 256 IPs

Private subnet CIDR block in us-east-1a

10.128.1.0/24 256 IPs

Private subnet CIDR block in us-east-1b

10.128.3.0/24 256 IPs

9. **NAT Gateways**: Select “**Zonal**” first and then “**1 per AZ**” in order.
10. **VPC Endpoints**: It doesn't matter whether you choose “**None**” or “**S3 Gateway**” since we will not use S3 buckets for current implementation.

NAT gateways (\$) - updated [Info](#)
NAT gateway allows private resources to access the internet from any availability zone within a VPC, providing a single managed internet exit point for the entire region. Additional charges apply.

None | Regional - new | **Zonal**

NAT gateways (\$) [Info](#)
Choose the number of Availability Zones (AZs) in which to create NAT gateways. Note that there is a charge for each NAT gateway.

In 1 AZ | **1 per AZ**

VPC endpoints [Info](#)
Endpoints can help reduce NAT gateway charges and improve security by accessing S3 directly from the VPC. By default, full access policy is used. You can customize this policy at any time.

None | **S3 Gateway**

DNS options [Info](#)

☒ Enable DNS hostnames

☒ Enable DNS resolution

11. Then proceed with “**Create VPC**” finally.

✔ Success

▼ **Details**

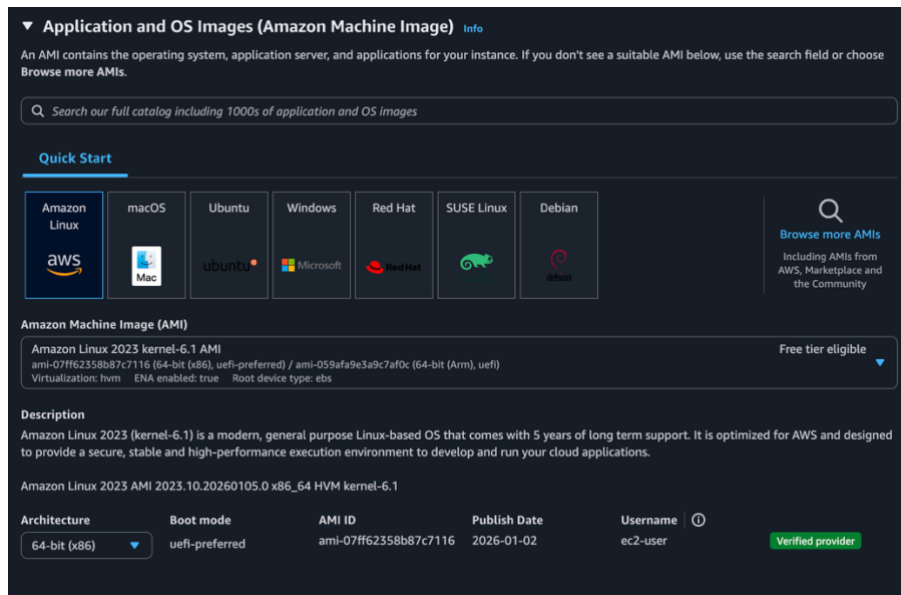
- ✔ Create VPC: vpc-02165b4d06e8f03df [↗](#)
- ✔ Enable DNS hostnames
- ✔ Enable DNS resolution
- ✔ Verifying VPC creation: vpc-02165b4d06e8f03df [↗](#)
- ✔ Create S3 endpoint: vpce-0506fde8df5533d34 [↗](#)
- ✔ Create subnet: subnet-0128327e5d5a6ebf2 [↗](#)
- ✔ Create subnet: subnet-0780a552e84e3cc8a [↗](#)
- ✔ Create subnet: subnet-0a6541e8c7474edb3 [↗](#)
- ✔ Create subnet: subnet-0bd2acbd658b9ead1 [↗](#)
- ✔ Create internet gateway: igw-05840fba2e0e8a9b8 [↗](#)
- ✔ Attach internet gateway to the VPC
- ✔ Create route table: rtb-002cc7947deafe0c6 [↗](#)
- ✔ Create route
- ✔ Associate route table
- ✔ Associate route table
- ✔ Allocate elastic IP: elipalloc-01ff61488138633d8 [↗](#)
- ✔ Allocate elastic IP: elipalloc-02d66c6e0a5fb7ef2 [↗](#)
- ✔ Create NAT gateway: nat-0d5677b67f0921b8 [↗](#)
- ✔ Create NAT gateway: nat-0a623aef776f7bc27 [↗](#)
- ✔ Wait for NAT Gateways to activate
- ✔ Create route table: rtb-0f78085370248deef [↗](#)
- ✔ Create route
- ✔ Associate route table
- ✔ Create route table: rtb-072cd1e205fd57f81 [↗](#)
- ✔ Create route
- ✔ Associate route table
- ✔ Verifying route table creation
- ✔ Associate S3 endpoint with private subnet route tables: vpce-0506fde8df5533d34 [↗](#)

EC2 Instance Creation

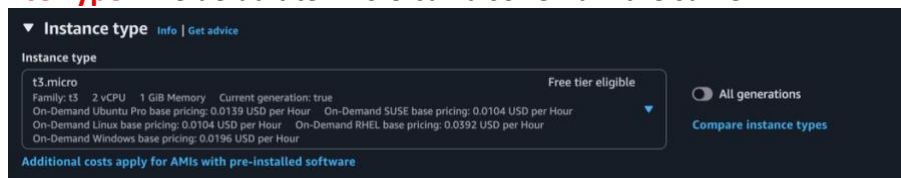
1. **Name:** Select any name you prefer, I named “wordpress-ec2”.



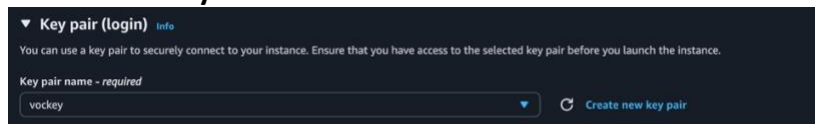
2. **Amazon Machine Image (AMI):** Do not need to change the default “Amazon Linux” and “Amazon Linux 2023 kernel-6.1 AMI” configuration and “64-bit (x86)” architecture.



3. **Instance Type:** The default t3.micro can also remain the same.



4. **Key Pair:** Select “vockey”.



5. **VPC:** Select the recently created VPC, “wordpress-vpc”.
6. **Subnet:** Choose any private subnet of given VPC, I choose us-east-1a private subnet “wordpress-subnet-private1-us-east-1a”.
7. **Auto-assign Public IP:** Disable the setting.
8. **Firewall (Security Group):** We do not have any security group yet so choose “Create Security Group”.
9. **Security Group Name:** Name any way you prefer, I named “wordpress-sg”.

▼ Network settings

Info

VPC - required

Info

vpc-02165b4d06e8f03df (wordpress-vpc)

10.128.0.0/16

Subnet

Info

subnet-0a65418c7474edb3

wordpress-subnet-private1-us-east-1a

VPC: vpc-02165b4d06e8f03df

Owner: 091781442554

Availability Zone: us-east-1a (use1-az2)

Zone type: Availability Zone

IP addresses available: 251

CIDR: 10.128.1.0/24

Create new subnet L2

Auto-assign public IP

Info

Disable

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

Security group name - required

wordpress-sg

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _~:/#.@!+=&().|!\$*

Description - required

Info

Security group for wordpress application

Inbound Security Group Rules

No security group rules are currently included in this template. Add a new rule to include it in the launch template.

Add security group rule

► Advanced network configuration

- 10.** Add a security group role as **Type** is “HTTP” and **Source Type** is “Anywhere”.

▼ Security group rule 2 (TCP, 80, 0.0.0.0/0)

Type Info	Protocol Info	Port range Info
HTTP	TCP	80
Source type Info	Source Info	Description - optional Info
Anywhere	Add CIDR, prefix list or security group	e.g. SSH for admin desktop
	0.0.0.0/0	

- 11. Configure Storage:** Can be kept in the default as “1x 8 GiB **gp3** Root volume, 3000 IOPS, not encrypted”.
- 12. Advanced Details:** It is important to choose **IAM Instance Profile** as “**LabInstanceProfile**”, so we can connect to EC2 with Session Manager easily on further sections.

▼

Advanced details

Info

Domain join directory

Info

Select

▼

IAM instance profile

Info

LabInstanceProfile

arn:aws:iam::091781442554:instance-profile/LabInstanceProfile

▼

- 13.** Then launch the EC2 instance.



Success

Successfully initiated launch of instance [\(i-0acc0e8be56cfe959\)](#)



Launch log

Initializing requests

 Succeeded

Creating security groups

 Succeeded

Launch initiation

 Succeeded

- 14.** When instance is launched, find the **“Target Groups”** in left scrollable area and click it, then in the top right click the **“Create Target Group”**. We will need a target group in Load Balancer.

15. **Target Type:** Keep the setting in **Instances**.
16. **Target Group Name:** Give a unique name for Target Group, I gave “**wordpress-tg**”.
17. **Protocol & Port:** Choose **HTTP** protocol and port **80**.

Settings - immutable
Choose a target type and the load balancer and listener will route traffic to your target. These settings can't be modified after target group creation.*

Target type
Indicate what resource type you want to target. Only the selected resource type can be registered to this target group.

☒ **Instances**
Supports load balancing to instances in a VPC. Integrate with Auto Scaling Groups or ECS services for automatic management.
Suitable for: **ALB** **NLB** **GWLB**

☐ **IP addresses**
Supports load balancing to VPC and on-premises resources. Facilitates routing to IP addresses and network interfaces on the same instance. Supports IPv6 targets.
Suitable for: **ALB** **NLB** **GWLB**

☐ **Lambda function**
Supports load balancing to a single Lambda function. ALB required as traffic source.
Suitable for: **ALB**

☐ **Application Load Balancer**
Allows use of static IP addresses and PrivateLink with an Application Load Balancer. NLB required as traffic source.
Suitable for: **NLB**

Target group name
Name must be unique per Region per AWS account.
wordpress-tg
Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters; Count: 12/32

Protocol
Protocol for communication between the load balancer and targets.
HTTP

Port
Port number where targets receive traffic. Can be overridden for individual targets during registration.
80
1-65535

18. **IP Address Type:** Keep in **IPv4**.
19. **VPC:** Select the previous VPC, “**wordpress-vpc**”.
20. **Protocol Version:** Keep on **HTTP1**.

IP address type
Only targets with the indicated IP address type can be registered to this target group.

☒ **IPv4**
Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

☐ **IPv6**
Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

VPC
Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.
vpc-02165b4d06e8f03df (wordpress-vpc)
10.128.0.0/16

Protocol version

☒ **HTTP1**
Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

☐ **HTTP2**
Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or gRPC, but gRPC-specific features are not available.

☐ **gRPC**
Send requests to targets using gRPC. Supported when the request protocol is gRPC.

21. Then proceed to “Next” page.
22. **Register Targets:** Select the recently created EC2 instance, which was “**wordpress-ec2**” for me. Then click “Include as pending below” button and proceed to “Next” page where you would review before clicking the “Create Target Group”.

Register targets - recommended
This is an optional step to create a target group. However, to ensure that your load balancer routes traffic to this target group you must register your targets.

Available instances (1/1)

Filter instances

Instance ID	Name	State	Security groups	Zone	Private IPv4 address
☒ i-0acc0e8be56cfe959	wordpress-ec2	Running	wordpress-sg	us-east-1a	10.128.1.24

1 selected

Ports for the selected instances
Ports for routing traffic to the selected instances.
80
1-65535 (separate multiple ports with commas)

[Include as pending below](#)

Review targets

Targets (1)

Filter targets ☐ Show only pending

Instance ID	Name	Port	State	Security groups	Zone	Private IPv4 address	Subnet ID	Launch time
i-0acc0e8be56cfe959	wordpress-ec2	80	Running	wordpress-sg	us-east-1a	10.128.1.24	subnet-0a6541e8c7474eddb3	January 21, 2026, 21:...

1 pending

[Cancel](#) [Previous](#) [Next](#)

Review and create
Review your target group configuration before creating

Step 1: Target group details [Edit](#)

Target group details			
Name wordpress-tg	Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1
VPC vpc-02165b4d06e8f03df i	IP address type IPv4		

Health check details			
Health check protocol HTTP	Health check path /	Health check port traffic-port	Interval 30 seconds
Timeout 5 seconds	Healthy threshold 5	Unhealthy threshold 2	Success codes 200

Step 2: Register targets [Edit](#)

Targets (1)				
Instance ID	Name	Port	Zone	
i-0acc0e8be56cfe959 i	wordpress-ec2	80	us-east-1a	

[Cancel](#) [Previous](#) [Create target group](#)

23. Find the “Load Balancer” in left scrollable area and click it, then in the top right click the “Create Load Balancer”.

24. Select Create from Application Load Balancer section in **Load Balancer Types**.

Load balancer types

Application Load Balancer [Info](#)

Choose an Application Load Balancer when you need a flexible feature set for your applications with HTTP and HTTPS traffic. Operating at the request level, Application Load Balancers provide advanced routing and visibility features targeted at application architectures, including microservices and containers.

[Create](#)

Network Load Balancer [Info](#)

Choose a Network Load Balancer when you need ultra-high performance, TLS offloading at scale, centralized certificate deployment, support for UDP, and static IP addresses for your applications. Operating at the connection level, Network Load Balancers are capable of handling millions of requests per second securely while maintaining ultra-low latencies.

[Create](#)

Gateway Load Balancer [Info](#)

Choose a Gateway Load Balancer when you need to deploy and manage a fleet of third-party virtual appliances that support GENEVE. These appliances enable you to improve security, compliance, and policy controls.

[Create](#)

► Classic Load Balancer - previous generation

25. **Load Balancer Name**: Give a unique name for Load Balancer, I gave “wordpress”.

26. **Scheme**: Select Internet-facing.

27. **Load Balancer IP Address Type**: Choose IPv4.

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.

wordpress

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme [Info](#)
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves internet-facing traffic.
- Has public IP addresses.
- DNS name resolves to public IPs.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name resolves to private IPs.
- Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type [Info](#)
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**

Includes only IPv4 addresses.

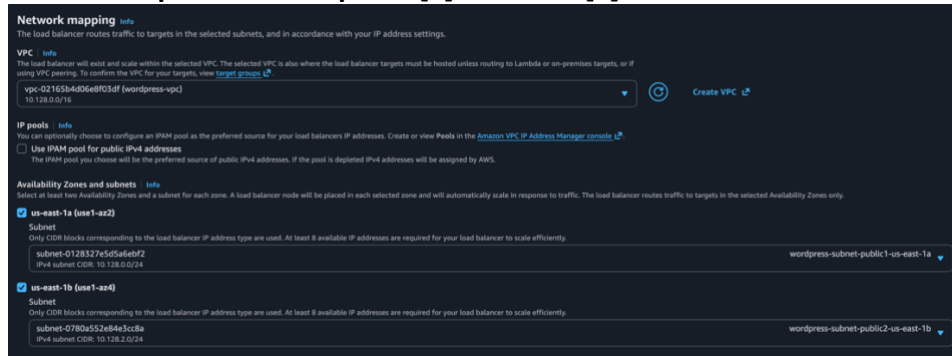
☐ **Dualstack**

Includes IPv4 and IPv6 addresses.

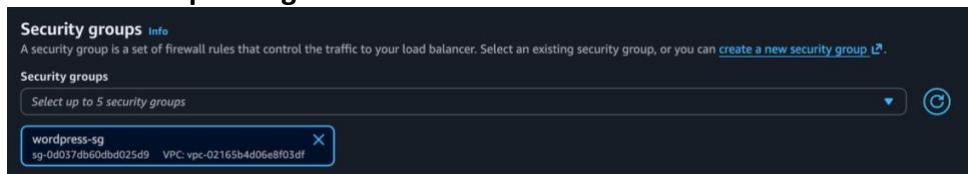
☐ **Dualstack without public IPv4**

Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with Internet-facing load balancers only.

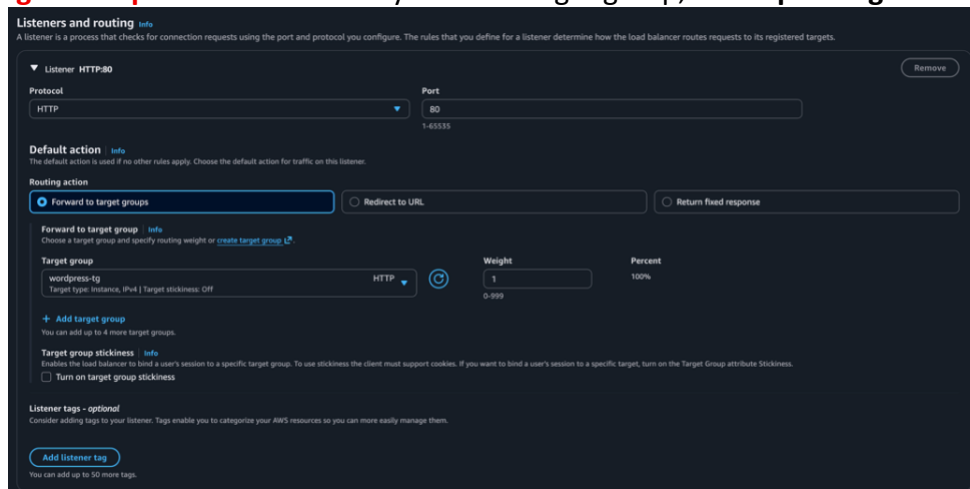
28. **VPC**: Select the previous VPC, “wordpress-vpc”.
29. **Availability Zones and Subnet**: Select both subnets and its corresponding public subnets “wordpress-subnet-public[x]-us-east-1[x]”.



30. **Security Groups**: Select the security group, which was created while creating VPC, for me it was “wordpress-sg”.



31. **Protocol & Port**: Choose HTTP protocol and port 80.
32. **Routing Action**: Select “Forward to target groups”.
33. **Target Group**: Select the recently created target group, “wordpress-tg”.

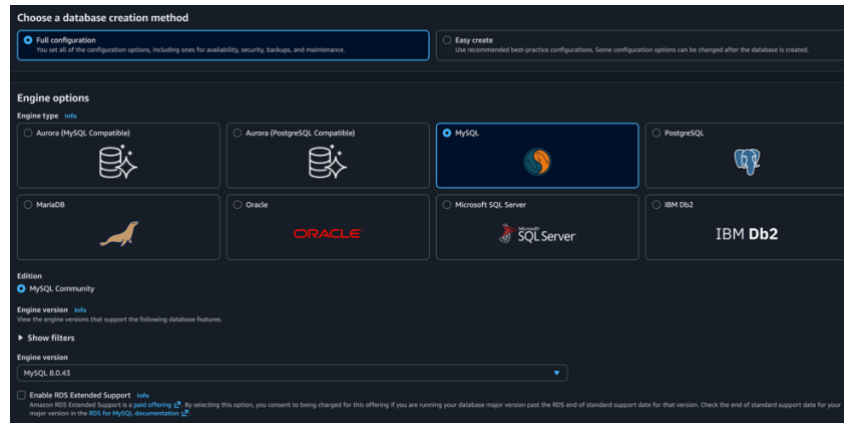


34. Then continue with “Create Load Balancer” which is located the end of page.

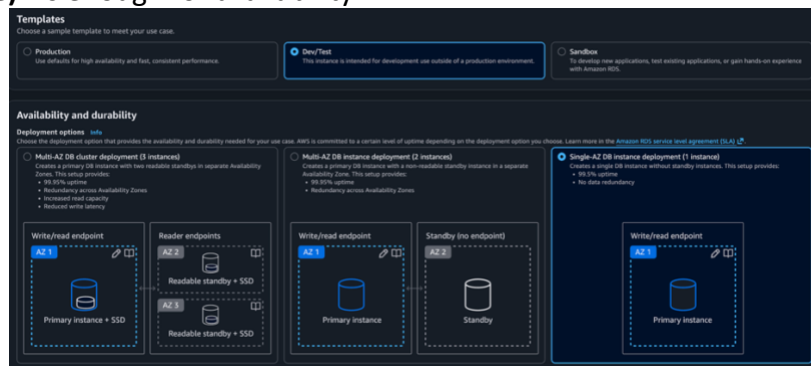


Aurora and RDS Configuration

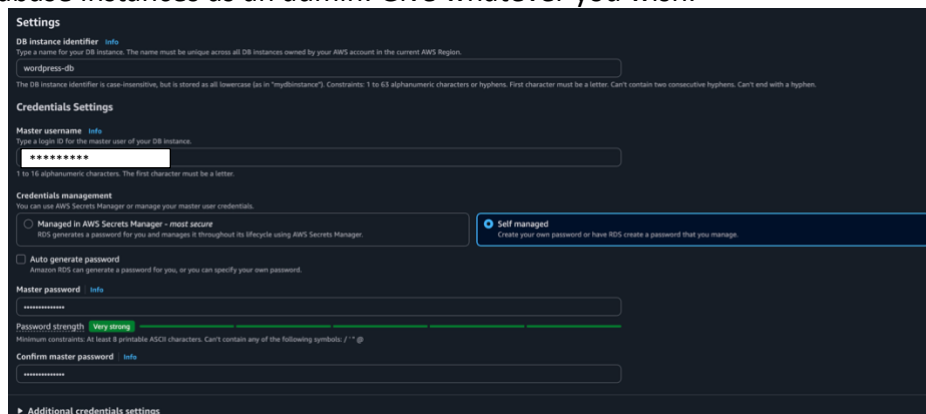
1. **Database Creation Method:** I choose **“Full Configuration”**, if detailed configurations are required.
2. **Engine Type:** Select **“MySQL”**.
3. **Engine Version:** Since I don't know the details and compatibility of MySQL versions, I kept it on default as **“MySQL 8.0.43”**, thinking as if a problem occurred I would use another version.



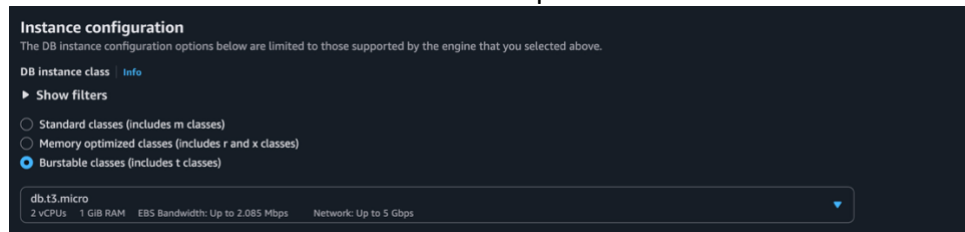
4. **Templates:** **“Dev/Test”** can be chosen since it is not a real-life project.
5. **Availability and Durability:** To save the budget, **“Single AZ-DB Instance Deployment (1 Instance)”** is enough for availability.



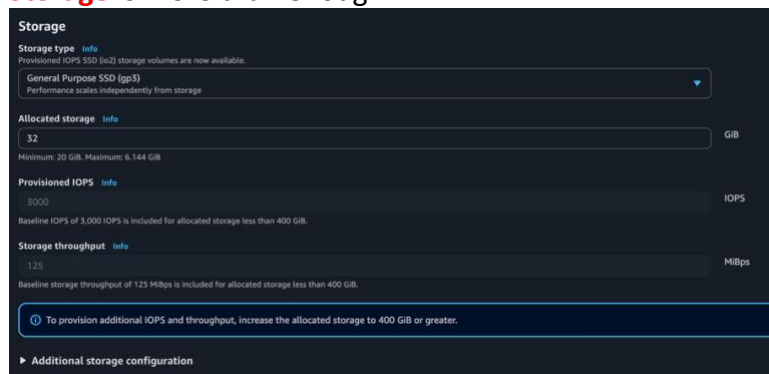
6. **DB Instance Identifier:** Define the name of database, I named it **“wordpress-db”**.
7. **Credential Settings:** In here you will define the username and password to access database instances as an admin. Give whatever you wish.



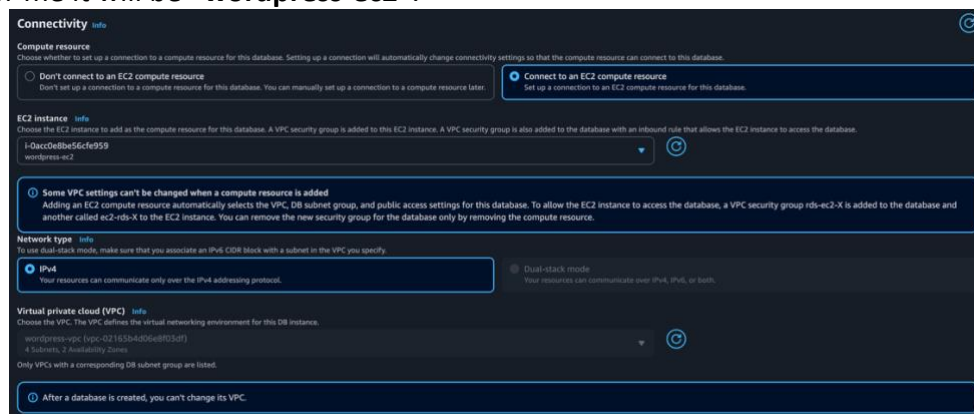
8. **Instance Configuration:** Most instances in RDS are pretty expensive. I checked the AWS calculator and online sources (e.g., <https://instances.vantage.sh/rds>) to decide, in conclusion “**db.t3.micro**” is one of the cheapest instances to use.



9. **Storage:** “**General Purpose SSD (gp3)**” can be used as **Storage Type** and **32 GiB** of **Allocated Storage** is more than enough.



10. **Connectivity:** We have to connect EC2 instance and RDS, so definitely choose the “**Connect to an EC2 Compute Resource**” option and select the regarding EC2 instance, for me it will be “**wordpress-ec2**”.



11. **DB Subnet Group:** We haven't create a DB subnet group, so choosing “**Automatic Setup**”, AWS will create and connect the subnets easily.
12. **Public Access:** Choose “**No**”.
13. **VPC Security Group (Firewall):** Select the existing security group, for me it will be “**wordpress-sg**”.

DB subnet group Info
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

☐ Choose existing
Choose existing DB subnet group

☒ Automatic setup
RDS creates a new subnet group for you or reuses an existing subnet group.

DB subnet group name
No VPC DB subnet groups
New DB subnet group created

Public access Info
☐ Yes
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.
☒ No
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

VPC security group (firewall) Info
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☒ Choose existing
Choose existing VPC security groups

☐ Create new
Create new VPC security group

Additional VPC security group
Choose one or more options

wordpress-ig X

14. **Availability Zone:** The option will be locked into the AZ that is defined on EC2 instance.

15. **Certificate Authority:** There is no need to change the default.

Availability Zone Info
us-east-1a

RDS Proxy
RDS Proxy is a fully managed, highly available database proxy that improves application scalability, resiliency, and security.
☐ Create an RDS Proxy Info
RDS automatically creates an IAM role and a Secrets Manager secret for the proxy. RDS Proxy has additional costs. For more information, see [Amazon RDS Proxy pricing](#).

Certificate authority - optional Info
Using a server certificate provides an extra layer of security by validating that the connection is being made to an Amazon database. It does so by checking the server certificate that is automatically installed on all databases that you provision.
rds-ca-rsa2048-g1 (default)
Expiry: May 26, 2061
If you don't select a certificate authority, RDS chooses one for you.

► Additional configuration

16. Monitoring options can be passed without configuring.

17. **Additional Configurations:** In here only, **Initial Database Name** is important and given. I named same with the DB Instance Identifier as **"wordpress_db"**. Backup and maintenance options are completely optional.

▼ **Additional configuration**
Database options, encryption turned on, backup turned on, backtrack turned off, maintenance, CloudWatch Logs, delete protection turned on.

Database options

Initial database name Info
wordpress_db

If you do not specify a database name, Amazon RDS does not create a database.

DB parameter group Info
default:mysql8.0

Option group Info
default:mysql-8.0

Backup

☒ Enable automated backup
Creates a point-in-time snapshot of your database.

Please note that automated backups are currently supported for InnoDB storage engine only. If you are using MyISAM, refer to details [here](#).

Backup retention period Info
The number of days (1-35) for which automatic backups are kept.
7 days

Backup window Info
The daily time range (in UTC) during which RDS takes automated backups.
☐ Choose a window
☒ No preference

Backup tags Info
Copies tags from the source database to the automated backup and snapshots respectively.
☐ Copy tags to automated backup
This is a one-time setting. Future tag modifications need manual updates.
☒ Copy tags to snapshots

Backup replication Info
☐ Enable replication in another AWS Region
Enabling replication automatically creates backups of your DB instance in the selected Region, for disaster recovery, in addition to the current Region.

☒ Enable encryption
Choose to encrypt the given instance. Master key IDs and aliases appear in the list after they have been created using the AWS Key Management Service console.

AWS KMS key Info
(default) aws/rds

Account
091781442554

KMS key ID
alias/aws/rds

Maintenance

Auto minor version upgrade Info
☒ Enable auto minor version upgrade
Enabling auto minor version upgrade will automatically upgrade your database minor version. For limitations and more details, see [Automatically upgrading the minor engine version documentation](#).

Maintenance window Info
Select the period you want pending modifications or maintenance applied to the database by Amazon RDS.
☐ Choose a window
☒ No preference

☒ Enable deletion protection
Protects the database from being deleted accidentally. While this option is enabled, you can't delete the database.

18. Then proceed with “Create Database” on bottom finally.

EC2 NGINX Configuration

1. Select the EC2 instance and click to the “Connect” button at the top-right corner.
2. Connect using “**Session Manager**”, since we defined IAM role on EC2 it shouldn’t be a problem.
3. Update the packages, using following;

```
sudo yum update -y
```

```
sudo dnf update -y
```

4. Download and run the nginx server, using following;

```
sudo dnf install nginx -y
```

```
sudo systemctl start nginx
```

```
sudo systemctl enable nginx
```

5. Download and run the required PHP and PHP modules, using following;

```
sudo dnf install php php-fpm php-mysqlnd -y
```

```
sudo systemctl start php-fpm
```

```
sudo systemctl enable php-fpm
```

6. Download and unzip the wordpress, using following;

```
sudo wget https://wordpress.org/latest.tar.gz
```

```
sudo tar -xzf latest.tar.gz
```

7. Erase the everything in the nginx’s base folder and load the folders from wordpress, using following;

```
sudo rm -rf /usr/share/nginx/html/*
```

```
sudo cp -r wordpress/* /usr/share/nginx/html/
```

8. Give permission for nginx to use the folders, using following;

```
sudo chown -R nginx:nginx /usr/share/nginx/html/
```

```
sudo chmod -R 755 /usr/share/nginx/html/
```

9. Open the nginx’s configuration file, using following;

```
sudo nano /etc/nginx/nginx.conf
```

10. Find the **server** block and;

- a. Add “**index index.php index.html index.htm;**” after **root** element,
- b. After adding index element add “**include /etc/nginx/default.d/*.conf;**”,
- c. Then after this two blocks add following;

```
location ~ \.php$ {  
    fastcgi_pass unix:/run/php-  
fpm/www.sock;  
    fastcgi_index index.php;  
    fastcgi_param SCRIPT_FILENAME  
$document_root$fastcgi_script_name;  
    include fastcgi_params;  
}
```

- d. Configured file should look like;

```
server {
    listen      80;
    listen      [::]:80;
    server_name _;
    root        /usr/share/nginx/html;
    index index.php index.html index.htm;
    include /etc/nginx/default.d/*.conf;

    location ~ \.php$ {
        fastcgi_pass unix:/run/php-fpm/www.sock;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
    }

    # Load configuration files for the default server block.
    include /etc/nginx/default.d/*.conf;

    error_page 404 /404.html;
    location = /404.html {
    }

    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
    }
}
```

- e. Save the file with “CTRL + O” and “Enter”.
f. Exit the file using “CTRL + X”.

11. Restart the nginx server, using following;

```
sudo systemctl restart nginx
```

12. Open the wordpress’s configuration file, using following;

```
cd /usr/share/nginx/html/
```

```
sudo cp wp-config-sample.php wp-config.php
```

```
sudo nano wp-config.php
```

13. Find the **database settings** comment and;

- a. Change the ‘**DB_NAME**’ value to your database name, mine was “wordpress_db”,
b. Change the ‘**DB_USER**’ value to your admin username,
c. Change the ‘**DB_PASSWORD**’ value to your admin password,
d. Change the ‘**DB_HOST**’ value to your Amazon RDS endpoint, mine was “wordpress-db.cdtkfyg2crta.us-east-1.rds.amazonaws.com”
e. Configured file should look like;

```
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress_db' );

/** Database username */
define( 'DB_USER', '*****' );

/** Database password */
define( 'DB_PASSWORD', '*****' );

/** Database hostname */
define( 'DB_HOST', 'wordpress-db.cdtkfyg2crta.us-east-1.rds.amazonaws.com' );
```

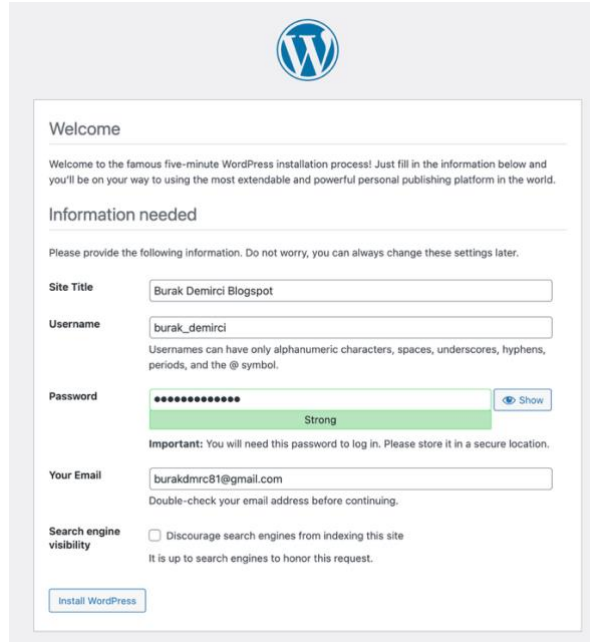
- f. Save the file with “CTRL + O” and “Enter”.
g. Exit the file using “CTRL + X”.

14. Then give permission to nginx to read, using following;

```
sudo chown -R nginx:nginx /usr/share/nginx/html/
```

Wordpress web should be accessible by the DNS name of the load balancer now!

Create an account and start creating your first post!



The image shows the WordPress installation welcome screen. At the top is the WordPress logo. Below it, a 'Welcome' message states: 'Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.' The 'Information needed' section follows, with a note: 'Please provide the following information. Do not worry, you can always change these settings later.' The form includes fields for 'Site Title' (filled with 'Burak Demirci Blogspot'), 'Username' (filled with 'burak_demirci'), 'Password' (filled with a strong password, with a 'Show' button), and 'Your Email' (filled with 'burakdemirci@gmail.com'). There is a checkbox for 'Search engine visibility' which is unchecked. At the bottom is an 'Install WordPress' button.

Yorucu Bir Günün Ardından 12 İzleme Önerisi

Written by [burak_demirci](#) in [eğlence](#)

İşten veya okuldan arda kalan boş vaktinizde sıkıldınız, izleyecek bir şeyler arıyorsunuz ve bulamadınız mı? Öyleyse size -izlemediğinizi var sayarak- her türden bir şeyler önereyim!

Kelimelerin Yetmediği Yerde: A Silent Voice

Tür: Dram, Psikolojik, Yaşamdan Kesitler (Slice of Life)

A Silent Voice, pişmanlık ve kendini affetme üzerine kurulmuş, izleyeni derinden sarsan sarsıcı bir başyapıt. Hikaye, ilkokulda işitme engelli sınıf arkadaşı Shoko'ya zorbalık yapan Shoya'nın, yıllar sonra duyduğu vicdan azabıyla girdiği telafi yolculuğunu konu alıyor. Film, iletişimin sadece seslerden ibaret olmadığını; işaret dili ve samimi bağlar üzerinden etkileyici bir dille anlatıyor. Karakterlerin iç dünyası ve büyüme sancıları o kadar şeffaf işlenmiş ki, her sahnede empati kurmaktan kendinizi alamıyorsunuz. Kyoto Animation'ın büyüleyici görselliği, bu hüzünlü ama umut dolu atmosferi adeta bir sanat eserine dönüştürüyor. Sadece bir animasyon değil, insanın ruhuna dokunan ve toplumsal dışlanmışlığı sorgulatan evrensel bir hikaye sunuyor. Eğer duygusal derinliği yüksek ve bittiğinde sizi uzun uzun düşündürecek bir film arıyorsanız, bu deneyimi kaçırmamalısınız.



The image is a still from the anime 'A Silent Voice'. It shows two characters, a boy and a girl, sitting on a bench under a large, blooming cherry blossom tree. The boy is on the left, looking towards the girl on the right. The girl has long red hair and is looking down. The scene is set in a lush, green environment with many cherry blossoms in the foreground and background.

Rest of the post can be read by:

<http://wordpress-529336691.us-east-1.elb.amazonaws.com/index.php/2026/01/23/yorucu-bir-gunun-ardindan-12-izleme-onerisi/>

BONUS

Here, it will be a short explanation for extra implementations for scaling.

Creating Amazon Machine Image (AMI)

1. Select the EC2 instance and click to the “Images and Templates” > “Create Image” button at the top-right corner.
2. **Image Name:** Give a name for your AMI, I named “wordpress-ec2-config”.

The screenshot shows the 'Create Image' page in the AWS Management Console. The 'Image details' section includes the 'Instance ID' (i-0acc0e8be56cfe959), 'Image name' (wordpress-ec2-config), and 'Image description - optional' (Configuration of wordpress-ec2). The 'Reboot instance' checkbox is checked. Below this is a table for 'Instance volumes' with columns for Storage type, Device, Snapshot, Size, Volume type, IOPS, Throughput, Delete on termination, and Encrypted. A note states: 'During the image creation process, Amazon EC2 creates a snapshot of each of the above volumes.' At the bottom, there are 'Tags - optional' fields with two radio buttons: 'Tag image and snapshots together' (selected) and 'Tag image and snapshots separately'.

3. Then proceed with “Create Image” at the bottom.

Creating Launch Template

1. Select the EC2 instance and click to the “Images and Templates” > “Create Template from Instance” button at the top-right corner.
2. **Launch Template Name:** Give a name for the launch template, I named “wordpress-launch-template”.

The screenshot shows the 'Create Template from Instance' page in the AWS Management Console. The 'Launch template name and description' section includes the 'Source instance' (i-0acc0e8be56cfe959), 'Launch template name - required' (wordpress-launch-template), and 'Template version description' (A prod webserver for MyApp). Below this is the 'Auto Scaling guidance' section with a checkbox 'Provide guidance to help me set up a template that I can use with EC2 Auto Scaling' which is checked. At the bottom, there is a 'Template tags' section with a right-pointing arrow.

3. **Application and OS Images (AMI):** Choose “My AMIs” in the top, and select the previously created AMI, which was “wordpress-ec2-config” for me.

▼ **Application and OS Images (Amazon Machine Image)** [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

AMI from catalog Recents **My AMIs** Quick Start

☐ Don't include in launch template ☒ Owned by me ☐ Shared with me

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

wordpress-ec2-config
ami-04a985d046fb11124
2026-01-23T19:20:55.000Z Virtualization: hvm ENA enabled: true Root device type: ebs Boot mode: uefi-preferred

Description

Architecture AMI ID

x86_64 ami-04a985d046fb11124

- Rest of the options should have copied from the main EC2 instance, so there is no need to change anything, but if a problem occurs here is how it is implemented into the rest for me,

▼ **Instance type** [Info](#) [Get advice](#) Advanced

Instance type

t3.micro Free tier eligible

Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour

On-Demand SUSE base pricing: 0.0104 USD per Hour On-Demand Linux base pricing: 0.0104 USD per Hour

On-Demand RHEL base pricing: 0.0392 USD per Hour On-Demand Windows base pricing: 0.0196 USD per Hour

☐ All generations [Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ **Key pair (login)** [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name

vockey Template value [Create new key pair](#)

▼ **Network settings** [Info](#)

Subnet [Info](#)

subnet-0a6541e8c7474edb3 wordpress-subnet-private1-us-east-1a

VPC: vpc-02165b4d06e8f03df Owner: 091781442554 Availability Zone: us-east-1a (use1-a2)

Zone type: Availability Zone IP addresses available: 249 CIDR: 10.128.1.0/24

[Create new subnet](#)

When you specify a subnet, a network interface is automatically added to your template.

Availability Zone [Info](#)

us-east-1a use1-a2 [Enable additional zones](#)

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Select existing security group ☐ Create security group

Common security groups [Info](#)

Select security groups

wordpress-sg sg-0d037db60dbd025d9 [Compare security group rules](#)

VPC: vpc-02165b4d06e8f03df

[Show all selected \(+1\)](#)

Security groups that you add or remove here will be added to or removed from all your network interfaces.

► **Advanced network configuration**

▼ **Storage (volumes)** [Info](#)

EBS Volumes [Hide details](#)

► Volume 1 (Template and AMI Root) : 8 GiB, EBS, General purpose SSD (gp3), 3000 IOPS

AMI Volumes are not included in the template unless modified

[Add new volume](#)

▼ **Resource tags** [Info](#)

No resource tags are currently included in this template. Add a resource tag to include it in the launch template.

[Add new tag](#)

You can add up to 50 more tags.

▼ **Advanced details** [Info](#)

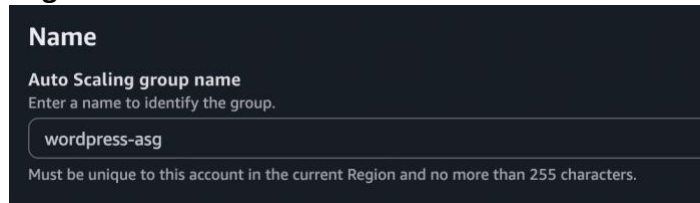
IAM instance profile [Info](#)

arn:aws:iam::091781442554:instance-profile/LabInstanceProfile

[Create new IAM profile](#)

Creating Auto Scaling Group

1. Find the “Auto Scaling Group” in left scrollable area and click it, then in the top right click the “Create Auto Scaling Group”.
2. **Auto Scaling Group Name:** Give a name for the auto scaling group, I named “wordpress-asg”.



Name

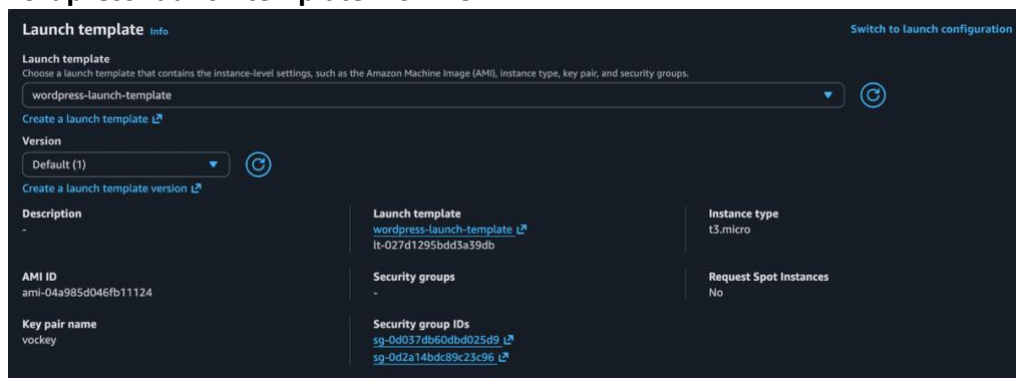
Auto Scaling group name

Enter a name to identify the group.

wordpress-asg

Must be unique to this account in the current Region and no more than 255 characters.

3. **Launch Template:** Select the previously created launch template, which was “wordpress-launch-template” for me.



Launch template [Info](#) [Switch to launch configuration](#)

Launch template

Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

wordpress-launch-template

Create a launch template [L](#)

Version

Default (1)

Create a launch template version [L](#)

Description

-

AMI ID

ami-04a985d046fb11124

Key pair name

vockey

Launch template

wordpress-launch-template [L](#)

lt-027d1295bdd3a39db

Instance type

t3.micro

Security groups

-

Request Spot Instances

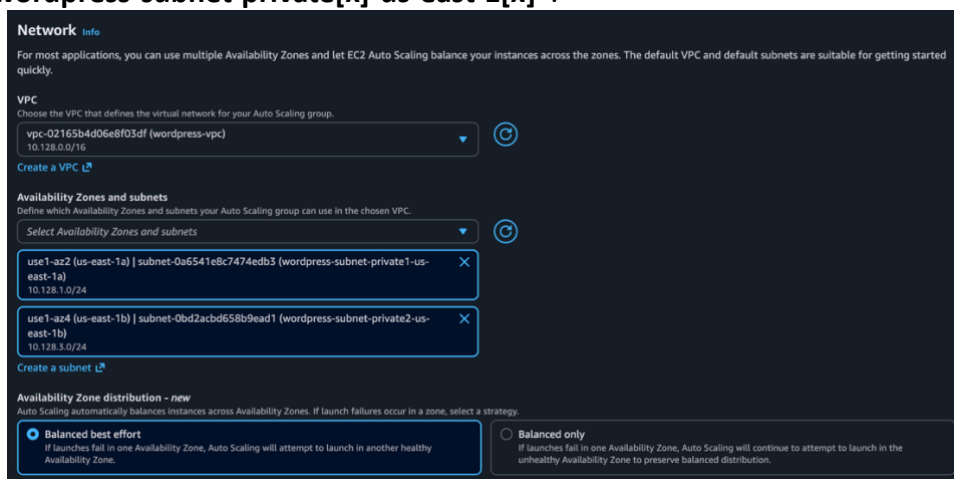
No

Security group IDs

sg-0d037db60bd025d9 [L](#)

sg-0d2a14bdc89c23c96 [L](#)

4. Then continue to next page.
5. **VPC:** Select the VPC created for Wordpress, which was “wordpress-vpc” for me.
6. **AZ and Subnets:** Choose the Private subnets in each AZ, which will look like “wordpress-subnet-private[x]-us-east-1[x]”.



Network [Info](#)

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC

Choose the VPC that defines the virtual network for your Auto Scaling group.

vpc-02165b4d06e8f03df (wordpress-vpc)

Create a VPC [L](#)

Availability Zones and subnets

Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

use1-az2 (us-east-1a) | subnet-0a6541e8c7474edb3 (wordpress-subnet-private1-us-east-1a)

use1-az4 (us-east-1b) | subnet-0bd2acbd658b9ead1 (wordpress-subnet-private2-us-east-1b)

Create a subnet [L](#)

Availability Zone distribution - new

Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

☒ Balanced best effort

If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

☐ Balanced only

If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

7. Then continue to next page.
8. **Load Balancing:** Select the existing load balancer using load balancer’s target group, which was “wordpress-tg” for me.

Load balancing Info
Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

Select Load balancing options

☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.

☒ **Attach to an existing load balancer**
Choose from your existing load balancers.

☐ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to an existing load balancer

Select the load balancers to attach

☒ Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.

☐ Choose from Classic Load Balancers

Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups

wordpress-tg | HTTP
Application Load Balancer: wordpress

9. Then continue to next page.

10. **Group Size & Scaling**: Select the whatever you would like to use, I used 1, 1, 2 respectively for desired, minimum and maximum.

Group size Info
Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity
Specify your group size.

1

Scaling Info
You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits
Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity
1
Equal or less than desired capacity

Max desired capacity
2
Equal or greater than desired capacity

11. Then proceed with “Next” until you reach the review page, then you can “**Create Auto Scaling Group**”.

12. Here’s a snapshot when the running main EC2 instance is stopped after the ASG creation;

Successfully initiated stopping of i-0acc0e8be56cfe959

Instances (2) Info Last updated less than a minute ago Connect Instance state ▼

Find Instance by attribute or tag (case-sensitive) All states ▼

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐		i-025b4ba4a77e99d87	Running	t3.micro	Initializing	View alarms +	us-east-1b
☐	wordpress-ec2	i-0acc0e8be56cfe959	Stopped	t3.micro	-	View alarms +	us-east-1a

P.S: EC2 instance configuration can be bugged when main instance (“wordpress-ec2” in this case) is stopped before ending the lab session for budget issues. If that occurs and website is not reachable when a new lab session is initiated, a new instance that is created by Auto Scaling Group solves the server connectivity problem with proper AMI configuration.